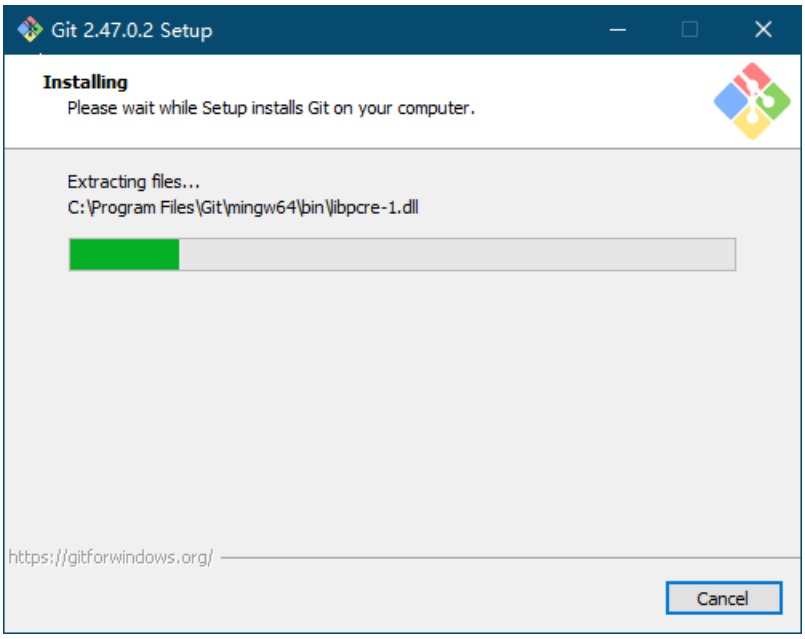
0x00 安装 git

https://git-scm.cn/install/windows 下载对应版本的 git 安装包并运行，选项很多，但是一路下一步就可以。



之后请参考《Gitee 操作极速上手指南（https://gitee.com/mvphp/gitee_yes）》进行配置。

0x01 安装 Visual Stuidio

https://visualstudio.microsoft.com/zh-hans/vs/ 下载 Visual Studio 安装包并运行。

请尽量按照下图选择安装选项。



0x02 安装 CMake

https://cmake.org/download/ 下载对应系统版本的安装包安装即可。

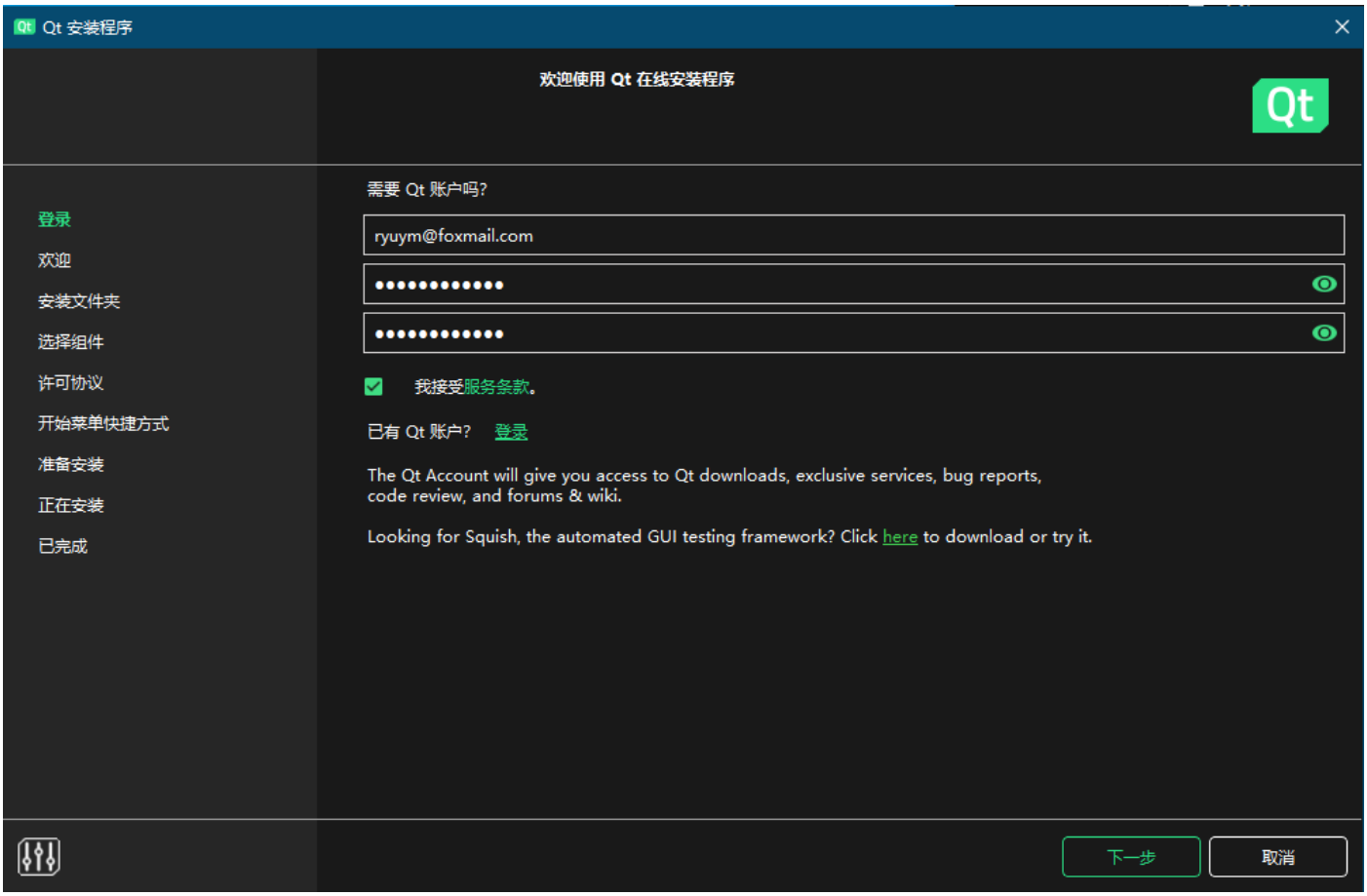
启动 Visual Studio，继续但无需代码，选项，CMake，常规，自定义 CMake 信息，勾选“启用自定义 CMake 可执行文件”，取消“对文件夹视图中的文件操作启用自动 CMake 脚本修改”，并设置 cmake.exe 的路径。



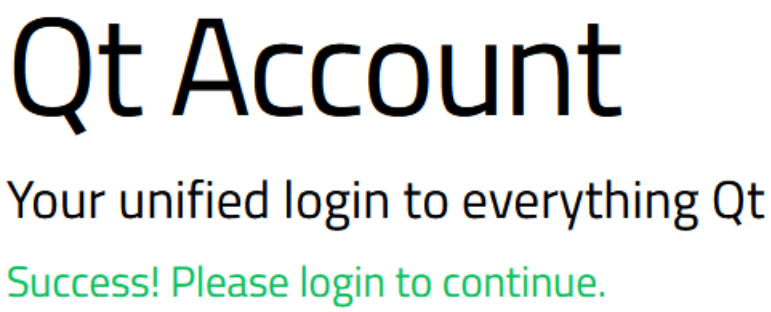
0x03 安装 Qt

https://download.qt.io/official_releases/online_installers/ 下载 qt-online-installer-windows-x64-online.exe，在其所在的文件夹打开命令行，运行 ./ qt-online-installer-windows-x64-online.exe --mirror https://mirrors.ustc.edu.cn/qtproject/ 进行安装。

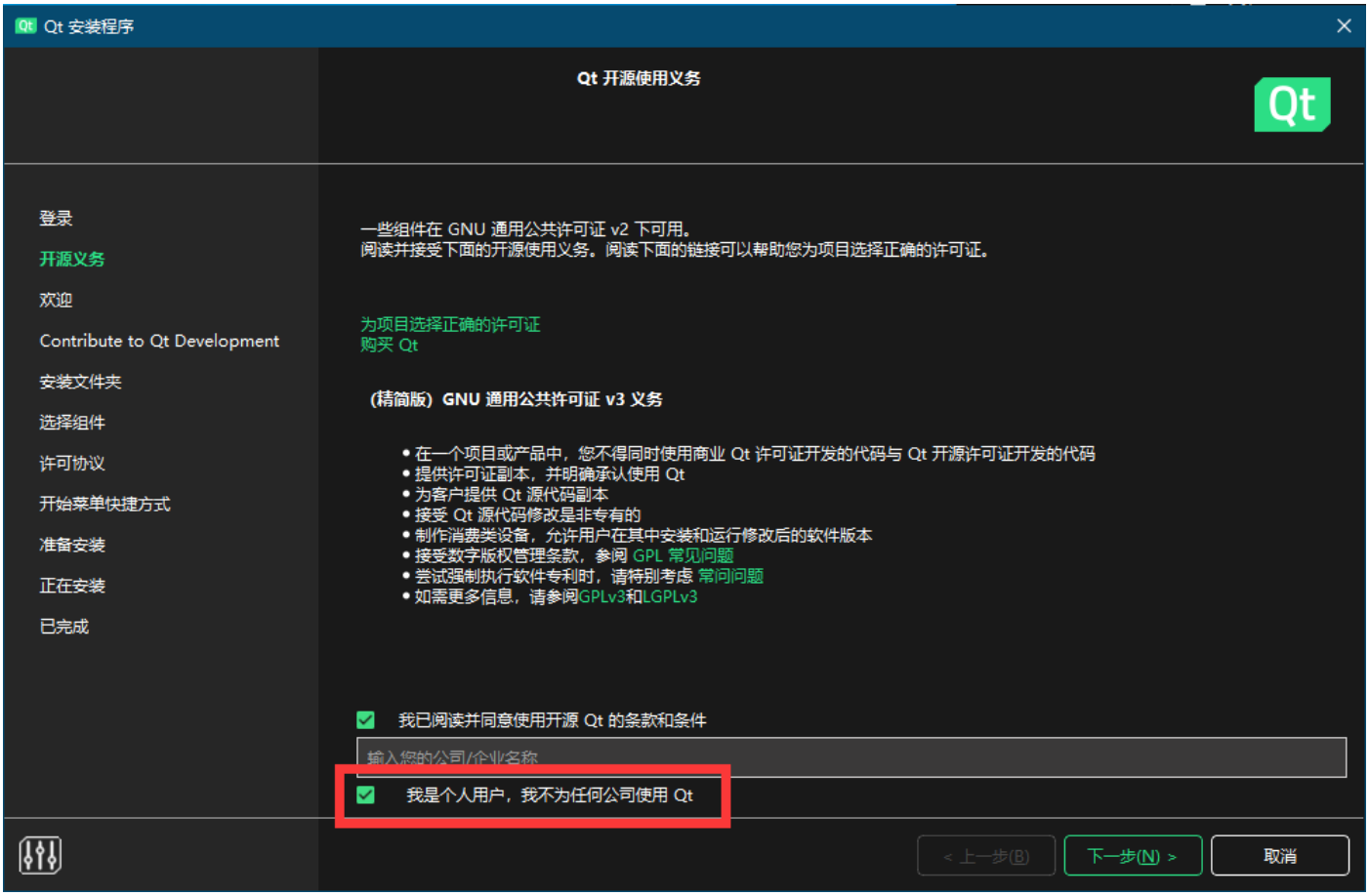
初次安装请注册 Qt 账号。



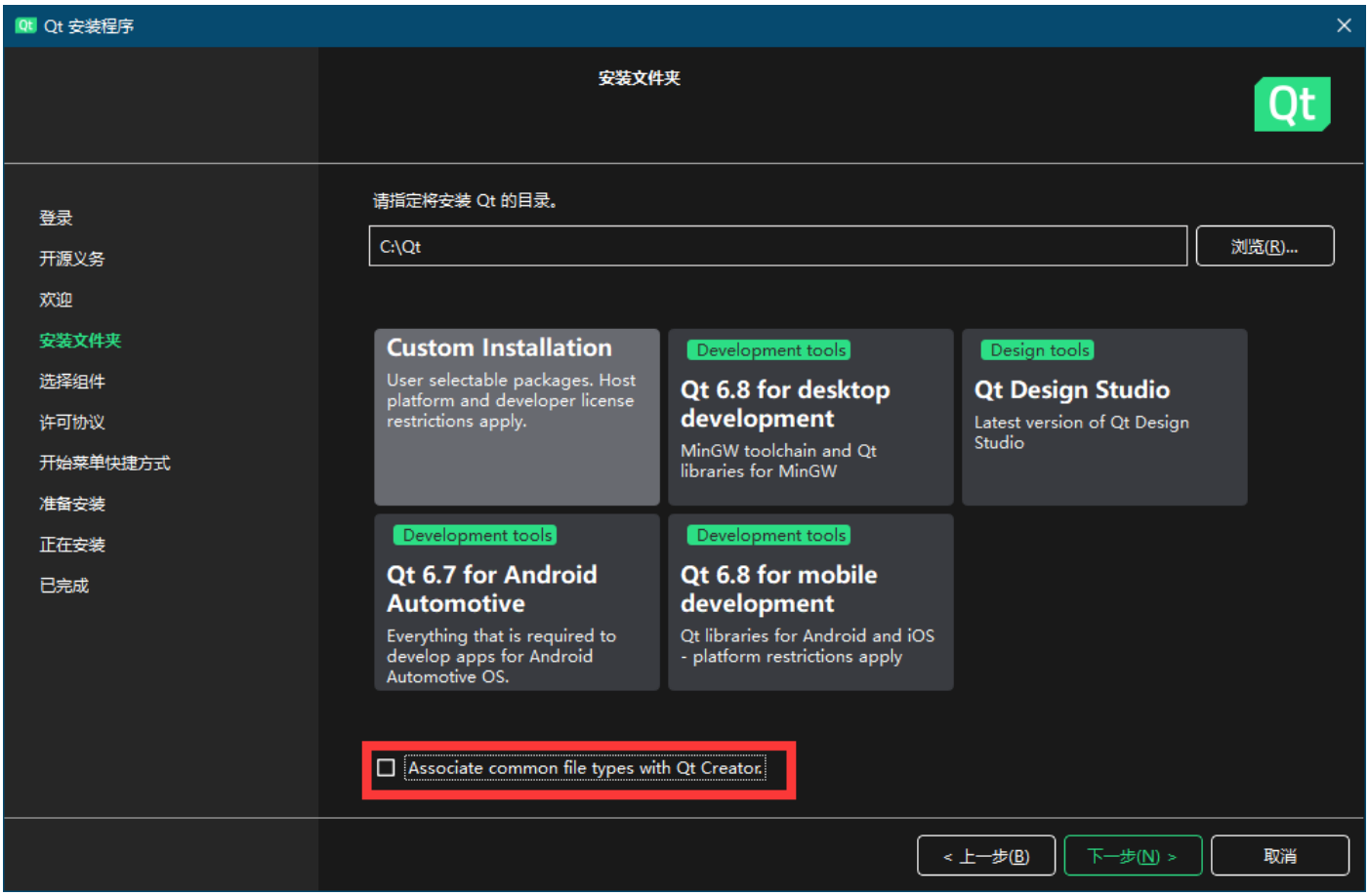
填写的邮箱中将会收到验证邮件，点开链接进入 Qt 网站填写个人信息。请务必勾选“I am an indicidual person not using Qt for any company”。



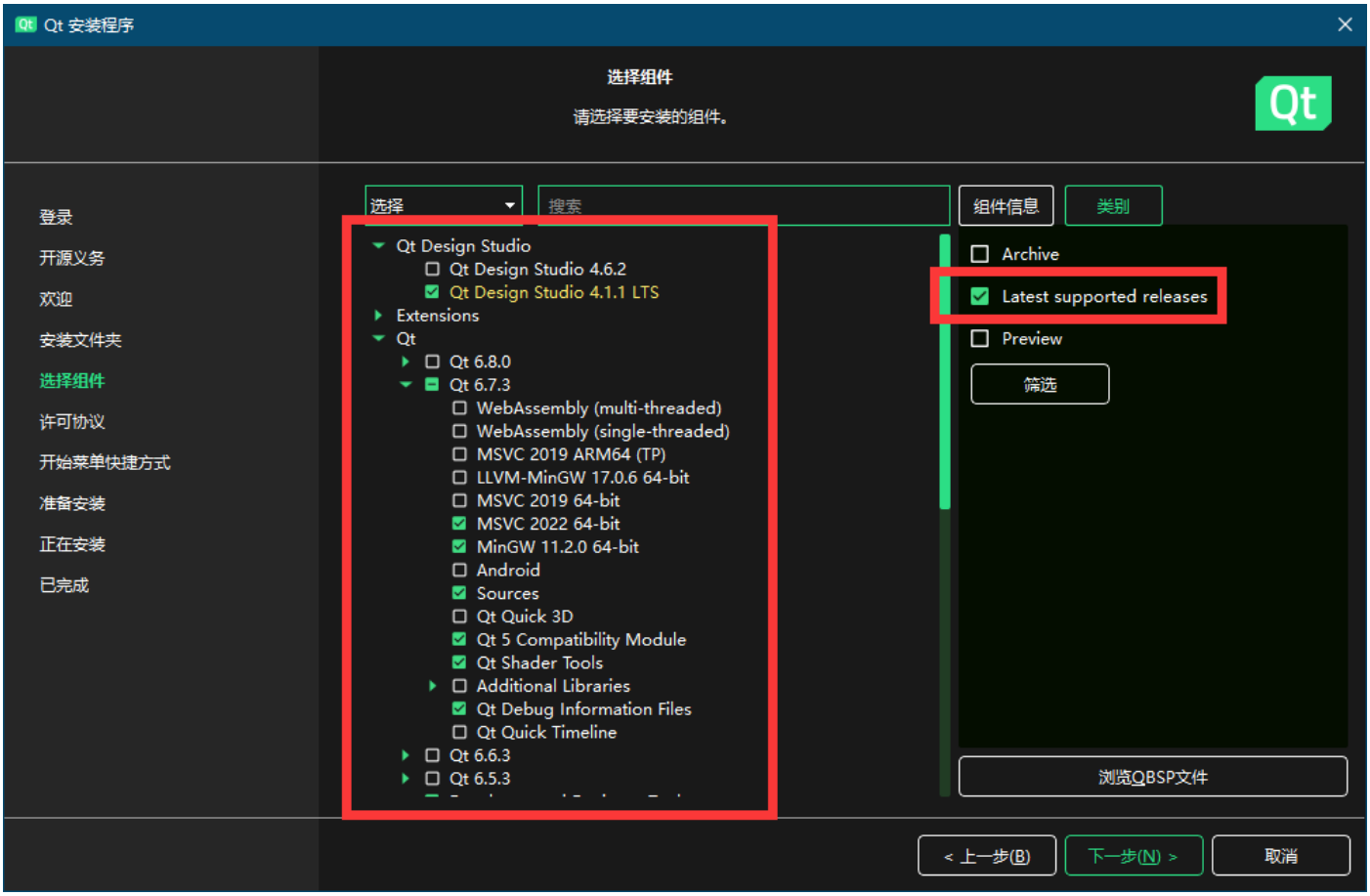
看到此页面即可回到 Qt 安装程序点击下一步。

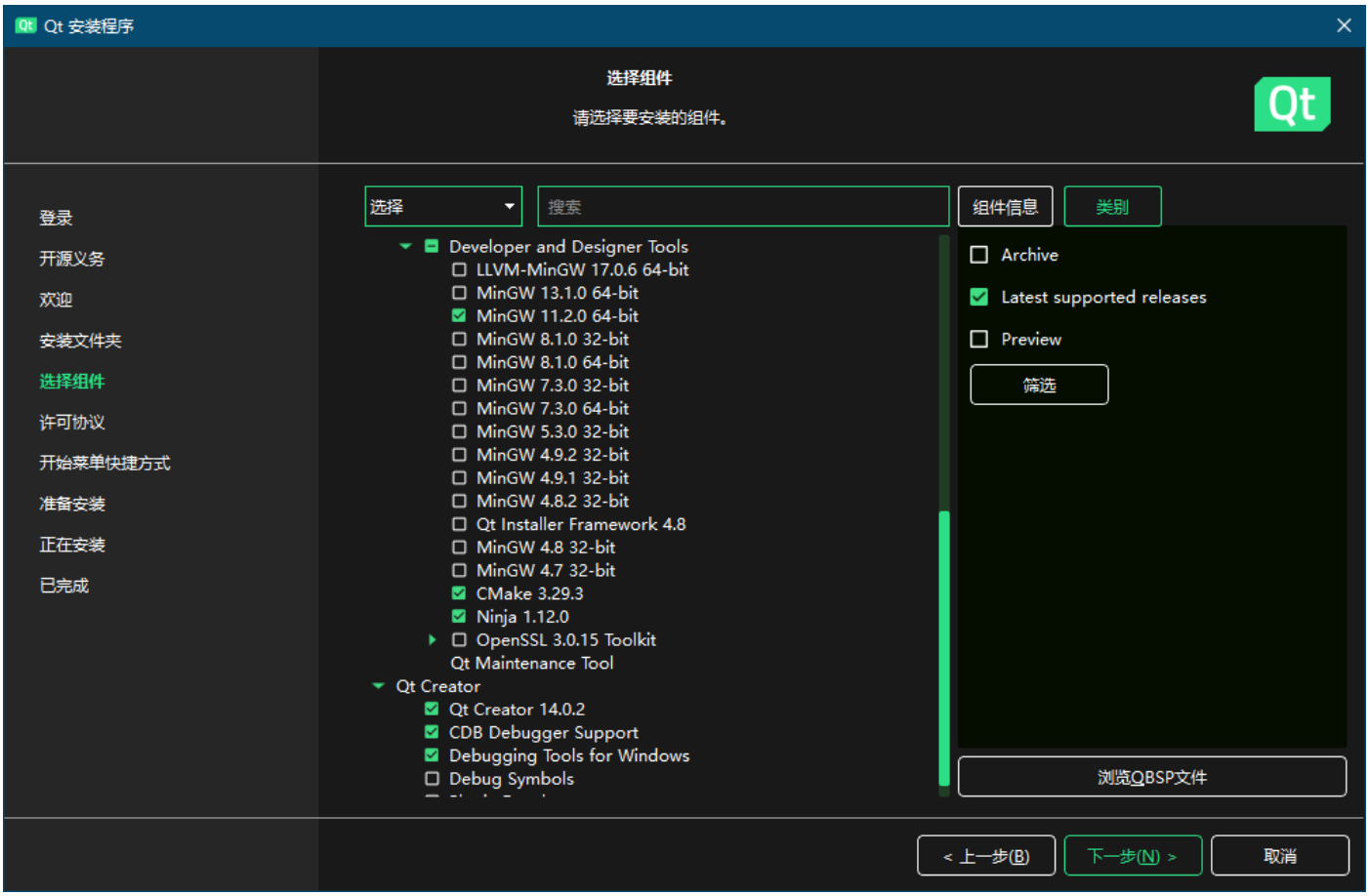


务必再次勾选“我是个人用户，我不为任何公司使用 Qt”。



之后选择组件，选择 Archive 与 Latest supported releases 类别，其中至少需要选择 Qt/Qt 6.7.3/ 的 MSVC 2022 64-bit 和 Sources 两项，其余组件根据需求选择。

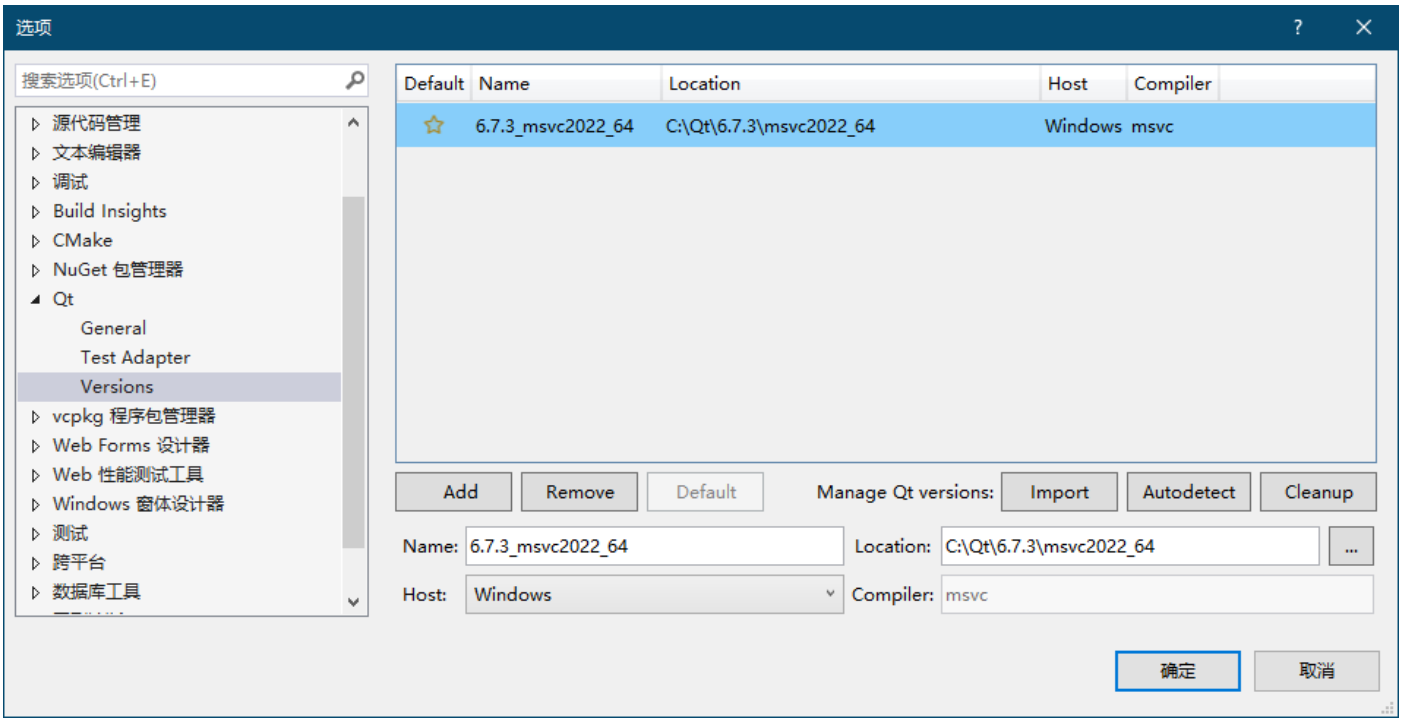




之后一路下一步即可。

https://marketplace.visualstudio.com/items?itemName=TheQtCompany.QtVisualStudioTools2022 下载并安装 Qt Visual Studio Tools 插件并安装。

运行 Visual Studio，继续但无需代码，扩展，Qt Vs Tools，Options，Version，直接 Autodetect。



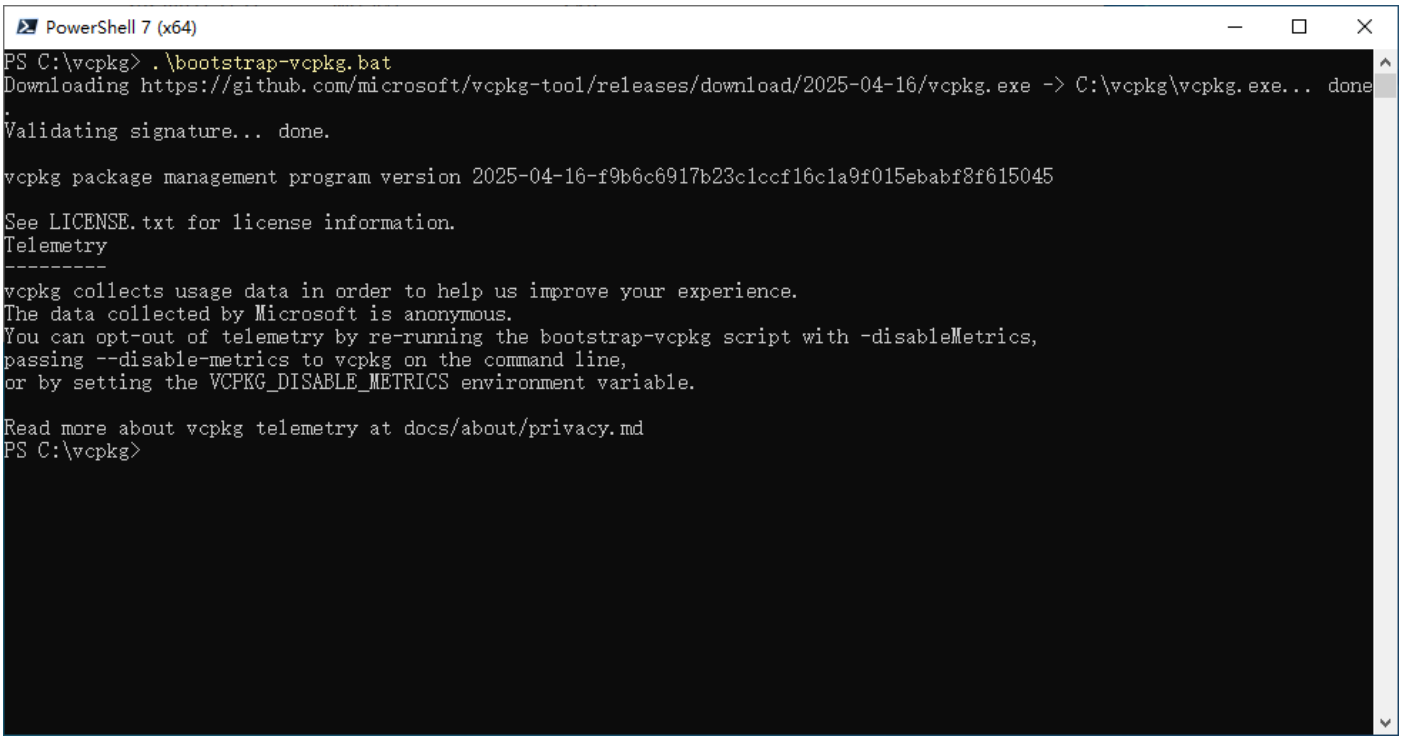
0x04 安装 vcpkg

https://github.com/PowerShell/PowerShell/releases 下载 PowerShell 安装包并进行安装，并将开始菜单中的 PowerShell 的 PowerShell 7 (x64) 移动到桌面上，并设置“默认以管理员模式运行此程序”，建议以后都通过 PowerShell 7 管理 vcpkg。

vcpkg 的文档参见 https://learn.microsoft.com/zh-cn/vcpkg/。

在 C:\ 打开 PowerShell，放在其他位置可能导致路径长度超过系统限制，运行 git clone https://github.com/microsoft/vcpkg.git。

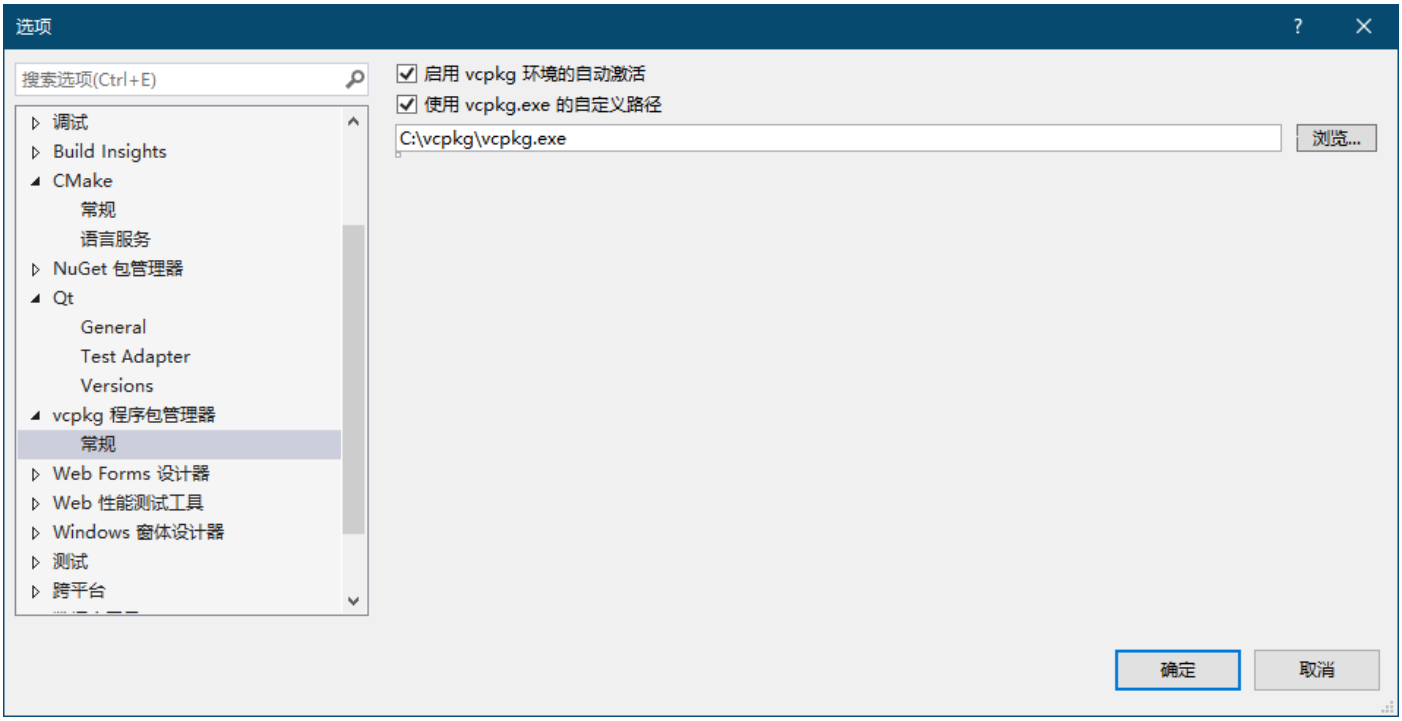
在 C:\vcpkg 打开 PowerShell，运行 .\bootstrap-vcpkg.bat，看到如下界面即安装成功。



将 C:\vcpkg 与 C:\vcpkg\installed\x64-windows\bin 与 C:\vcpkg\installed\x64-windows\debug\bin 添加到环境变量的系统变量的 Path 中。

运行桌面的 PowerShell 7 (x64)，输入 vcpkg search fmt 并执行，若显示出可供安装的 fmt 库，则成功安装 vcpkg。

启动 Visual Studio，继续但无需代码，选项，vcpkg 程序包管理器，常规，勾选“使用 vcpkg.exe 的自定义路径”，并设置 vcpkg.exe 的路径。

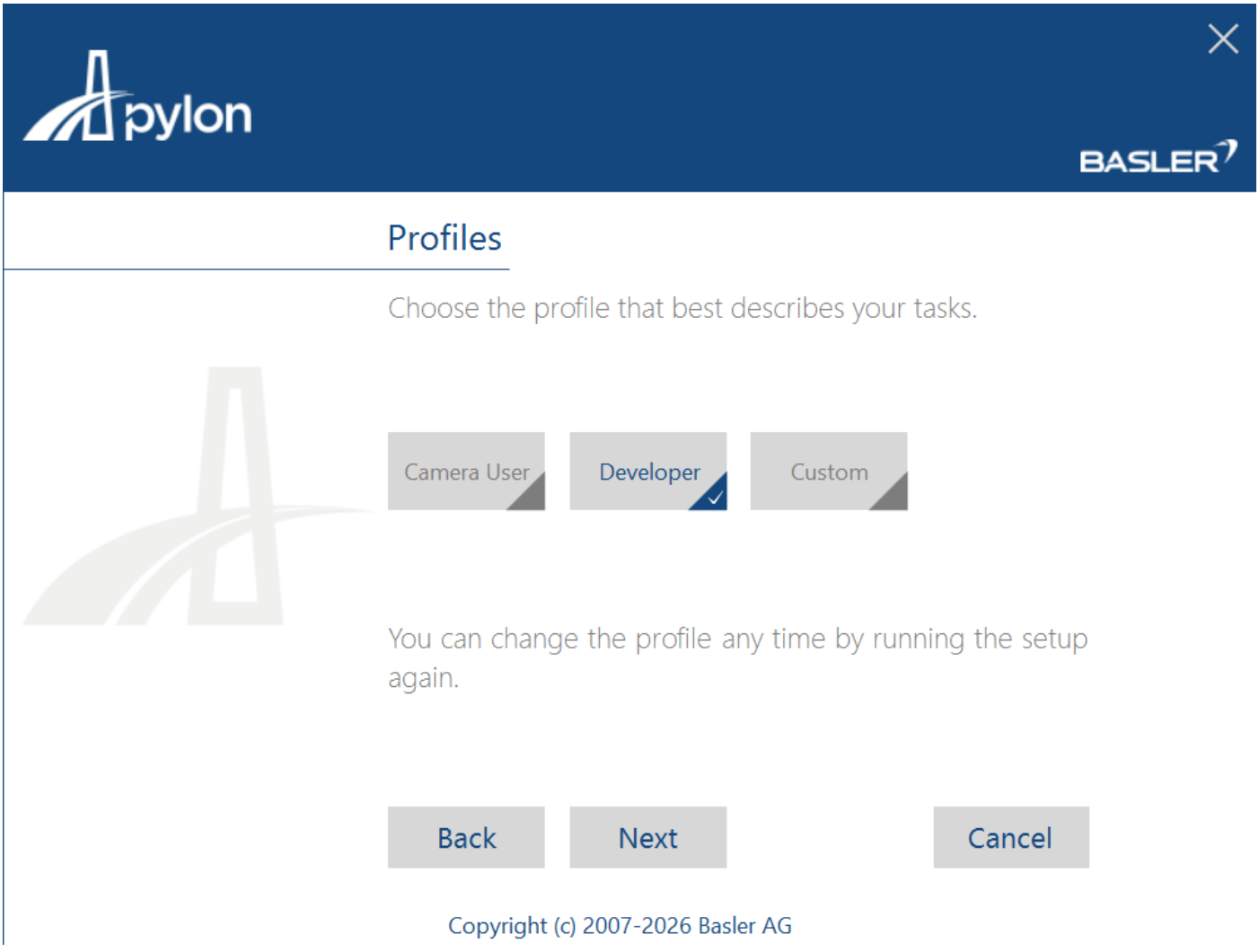

选项
搜索选项(Ctrl+E)
调试
Build Insights
CMake
常规
语言服务
NuGet 包管理器
Qt
General
Test Adapter
Versions
vcpkg 程序包管理器
常规
Web Forms 设计器
Web 性能测试工具
Windows 窗体设计器
测试
跨平台
启用 vcpkg 环境的自动激活
使用 vcpkg.exe 的自定义路径
C:\vcpkg\vcpkg.exe
浏览...
确定
取消

0x05 安装 basler SDK

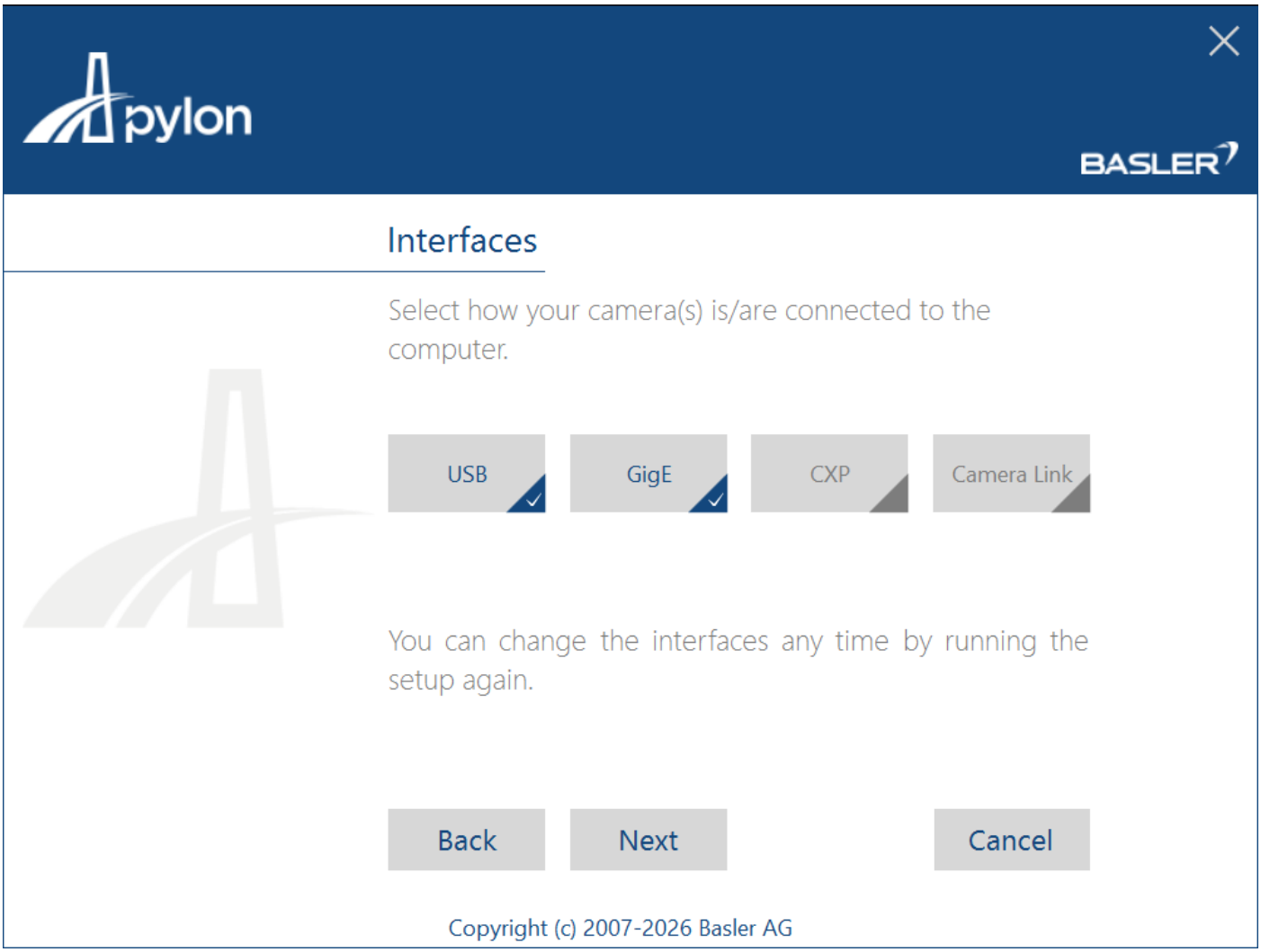
https://www.baslerweb.cn/zh-cn/downloads/software/ 下载 pylon 相机软件套装并以管理员身份运行。



profile 选择 Developer。



相机接口根据实验室需求选择。



安装位置不建议改动，之后一路下一步安装即可。

安装完成后请确认，系统环境变量的系统变量中新增：

```
CMAKE_PREFIX_PATH: C:\Program Files\Basler\pylon\Development\CMake\

GENICAM_GENTL32_PATH: C:\Program Files\Basler\pylon\Runtime\Win32\

GENICAM_GENTL64_PATH: C:\Program Files\Basler\pylon\Runtime\x64\

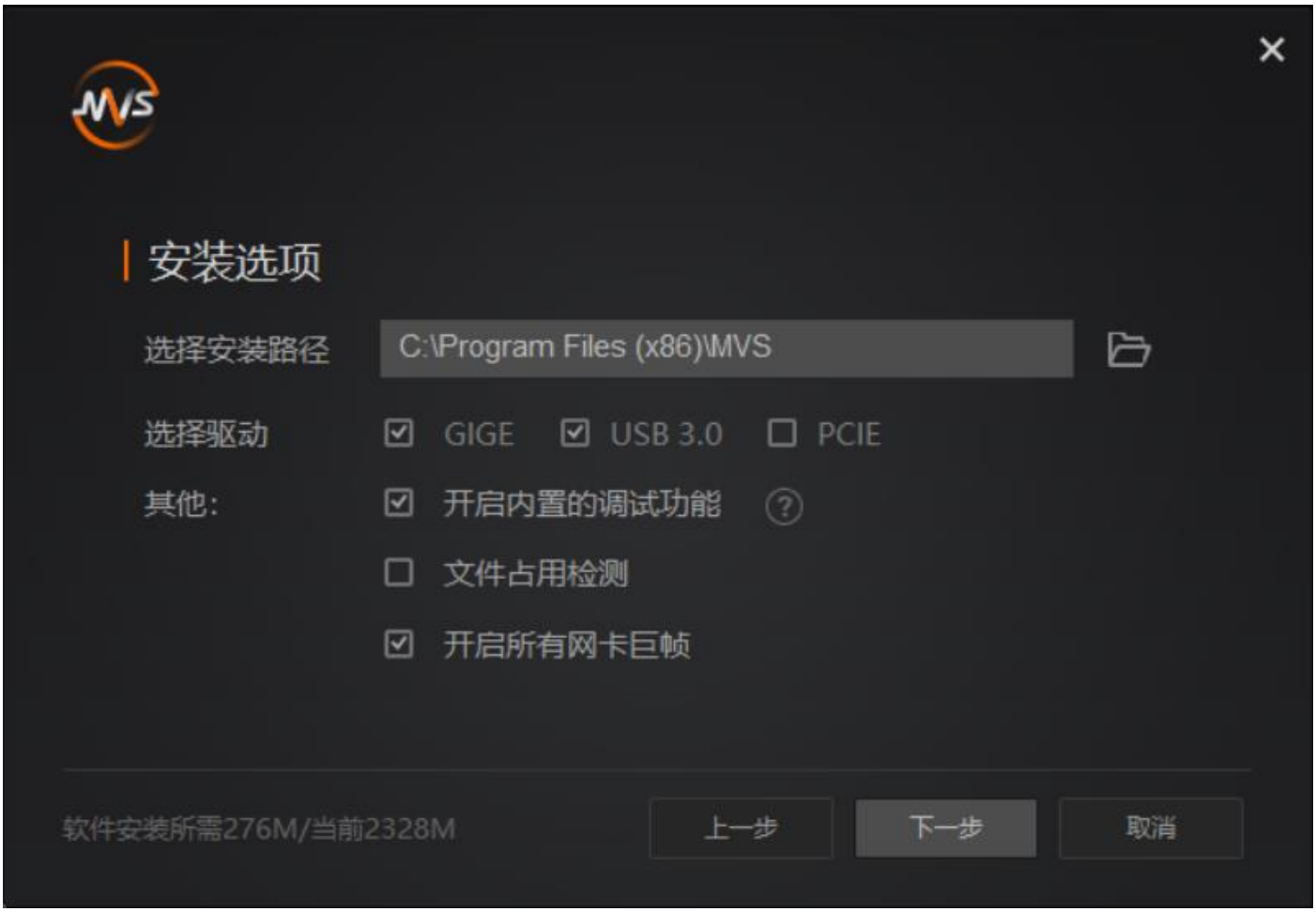
Path: C:\Program Files\Basler\pylon\Runtime\x64\

Path: C:\Program Files\Basler\pylon\Runtime\Win32\
```

0x06 安装海康 SDK

https://www.hikrobotics.com/cn/machinevision/service/download/ 下载机器视觉工业相机客户端 MVS 并安装。





安装路径不建议改动，驱动根据实验室需求选择。

安装完成后请确认，系统环境变量的系统变量中新增：

GENICAM_GENTL32_PATH: C:\Program Files (x86)\Common Files\MVS\Runtime\Win32_i86

GENICAM_GENTL64_PATH: C:\Program Files (x86)\Common Files\MVS\Runtime\Win64_x64

MVCAM_COMMON_RUNENV: C:\Program Files (x86)\MVS\Development

MVCAM_GENICAM_CLPROTOCOL: C:\Program Files (x86)\Common

Files\MVS\Runtime\CLProtocol

MVCAM_TOOLKIT: C:\Program Files (x86)\MvToolkit

Path: C:\Program Files (x86)\Common Files\MVS\Runtime\Win32_i86

Path: C:\Program Files (x86)\Common Files\MVS\Runtime\Win64_x64

0x07 安装大恒 SDK

https://www.daheng-imaging.com/downloads/softwares/ 下载 Galaxy Windows SDK_CN 并安装。





安装位置不建议改动，之后一路下一步安装即可。

安装完成后请确认，系统环境变量的系统变量中新增：

GALAXY_GENICAM_ROOT: C:\Program Files\Daheng Imaging\GalaxySDK\GenICam

GALAXY_SDK_DEVELOPMENT: C:\Program Files\Daheng Imaging\GalaxySDK\Development

GALAXY_SDK_DRIVER: C:\Program Files\Daheng Imaging\GalaxySDK\Drivers\

GENICAM_GENTL32_PATH: C:\Program Files\Daheng Imaging\GalaxySDK\GenTL\Win32

GENICAM_GENTL64_PATH: C:\Program Files\Daheng Imaging\GalaxySDK\GenTL\Win64

Path: C:\Program Files\Daheng Imaging\GalaxySDK\GenICam\bin\Win64_x64

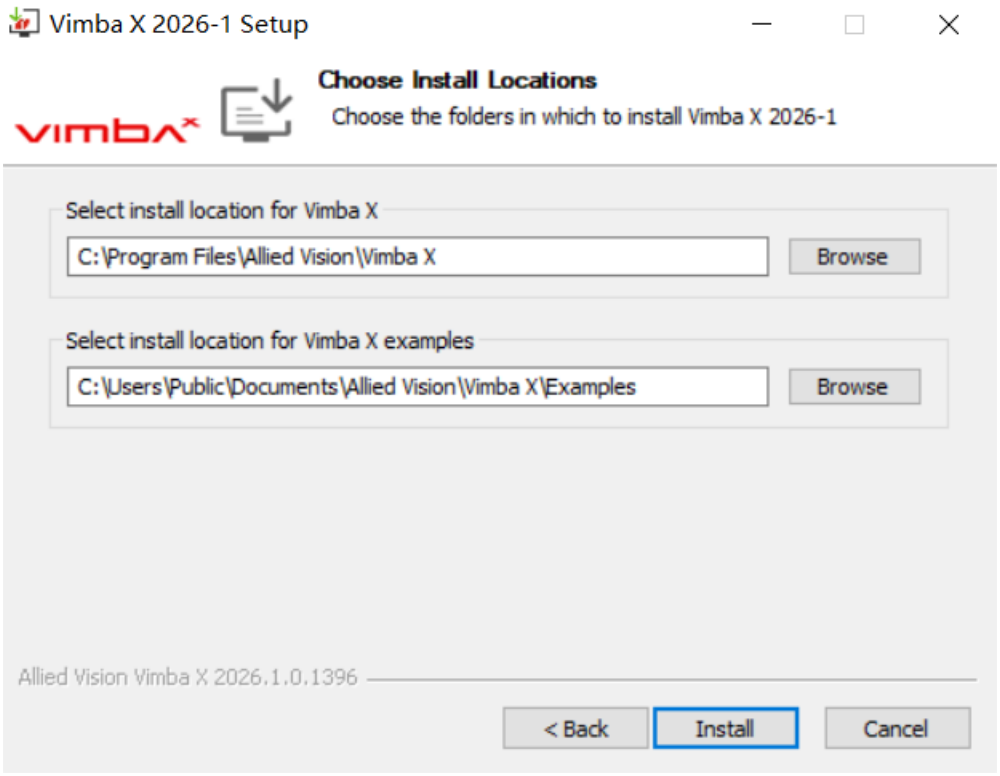
Path: C:\Program Files\Daheng Imaging\GalaxySDK\APIDll\Win64

Path: C:\Program Files\Daheng Imaging\GalaxySDK\GenICam\bin\Win32_i86

Path: C:\Program Files\Daheng Imaging\GalaxySDK\APIDll\Win32

0x08 安装 Allied SDK

https://www.alliedvision.cn/cn/support/software-downloads/vimba-x-sdk/vimba-x 下载 Vimba X 并安装，安装之前请确保相机已经连接电脑。



安装位置不建议改动，之后一路下一步安装即可。

安装完成后请确认，系统环境变量的系统变量中新增：

```
VIMBA_X_HOME: C:\Program Files\Allied Vision\Vimba X
```

并手动在系统变量中新增：

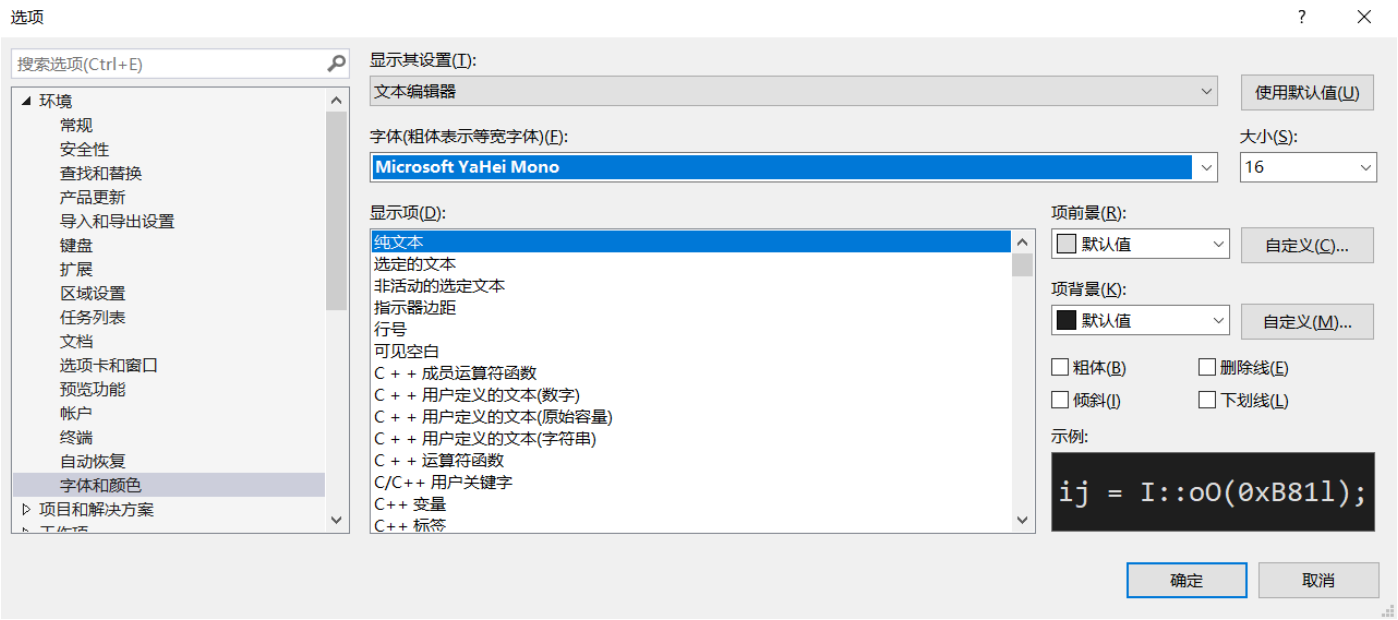
```
Path: C:\Program Files\Allied Vision\Vimba X\api\bin
```

0x09 设置 Visual Studio

https://marketplace.visualstudio.com/items?itemName=qazwsxlty.forceutf8nobom2022 下载并安装 Force UTF-8 (No BOM) 2022 插件，此扩展用于强制使用 UTF-8 编码。

从 https://github.com/yakumioto/YaHei-Consolas-Hybrid-1.12 或 https://github.com/chenyium/Microsoft-Yahei-Mono 中选择一款字体下载并安装，两款字体都是经过验证的适合中英文混合编程的字体。

启动 Visual Studio，继续但无需代码，选项，环境，字体和颜色，选择安装的字体即可。



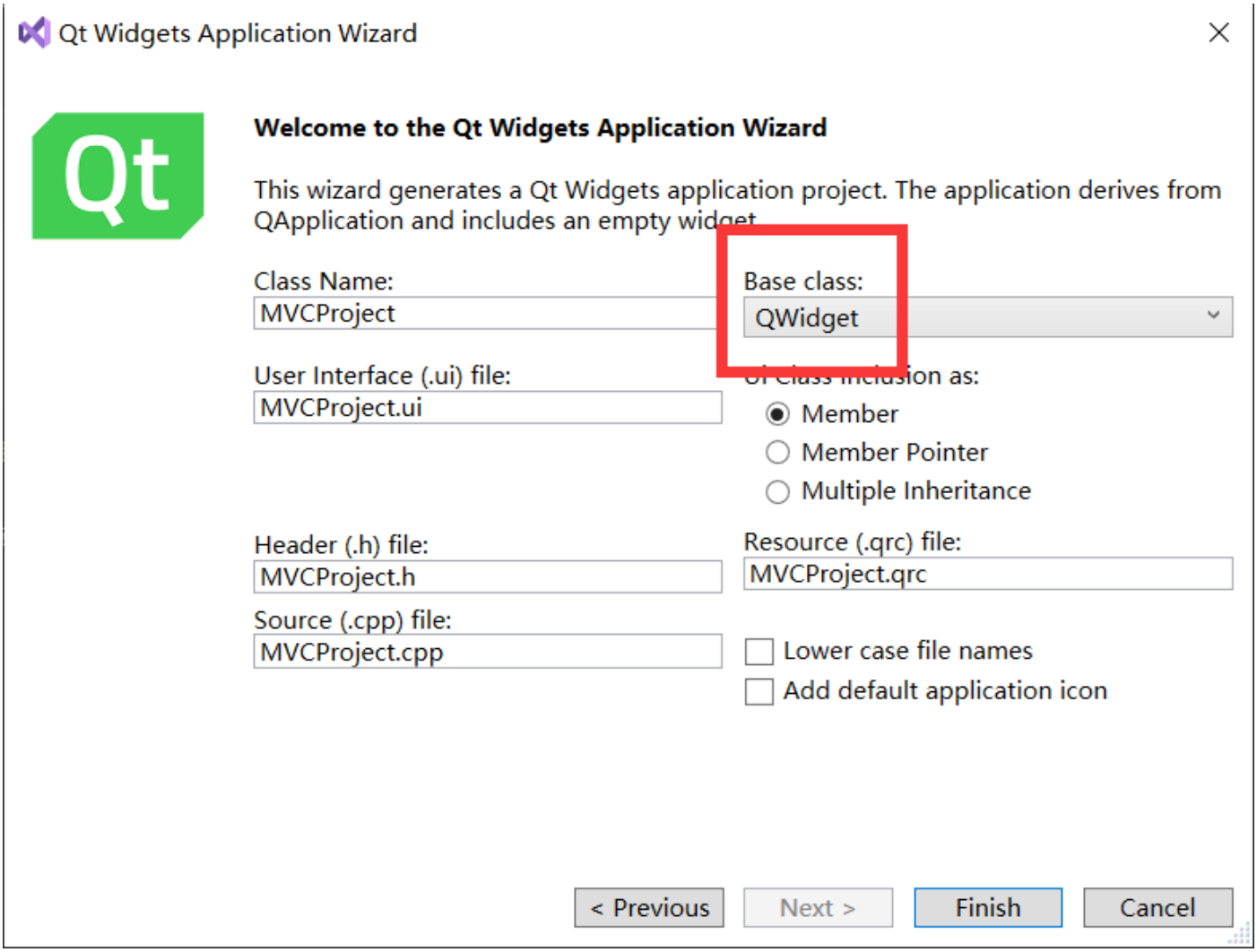
另外，调整字体大小请尽量通过调整字号的方式进行，而非“Ctrl+滚轮”进行整体缩放，从而保证最佳的阅读体验。

为了解决编译完成后运行前，闪过命令行提示：Neither dumpbin, llvm-objdump nor objdump could be found. Can not take care of dll dependencies. 的问题，将 dumpbin.exe 的路径添加到系统变量的 Path 中，通常位于 C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\MSVC\14.44.35207\bin\Hostx64\x64。

0x09 测试一：创建与 vcpkg 联动的 Qt 新项目

运行 Visual Studio 2022，创建新项目，选择 Qt Widgets Application，填写项目名称与路径进行创建。

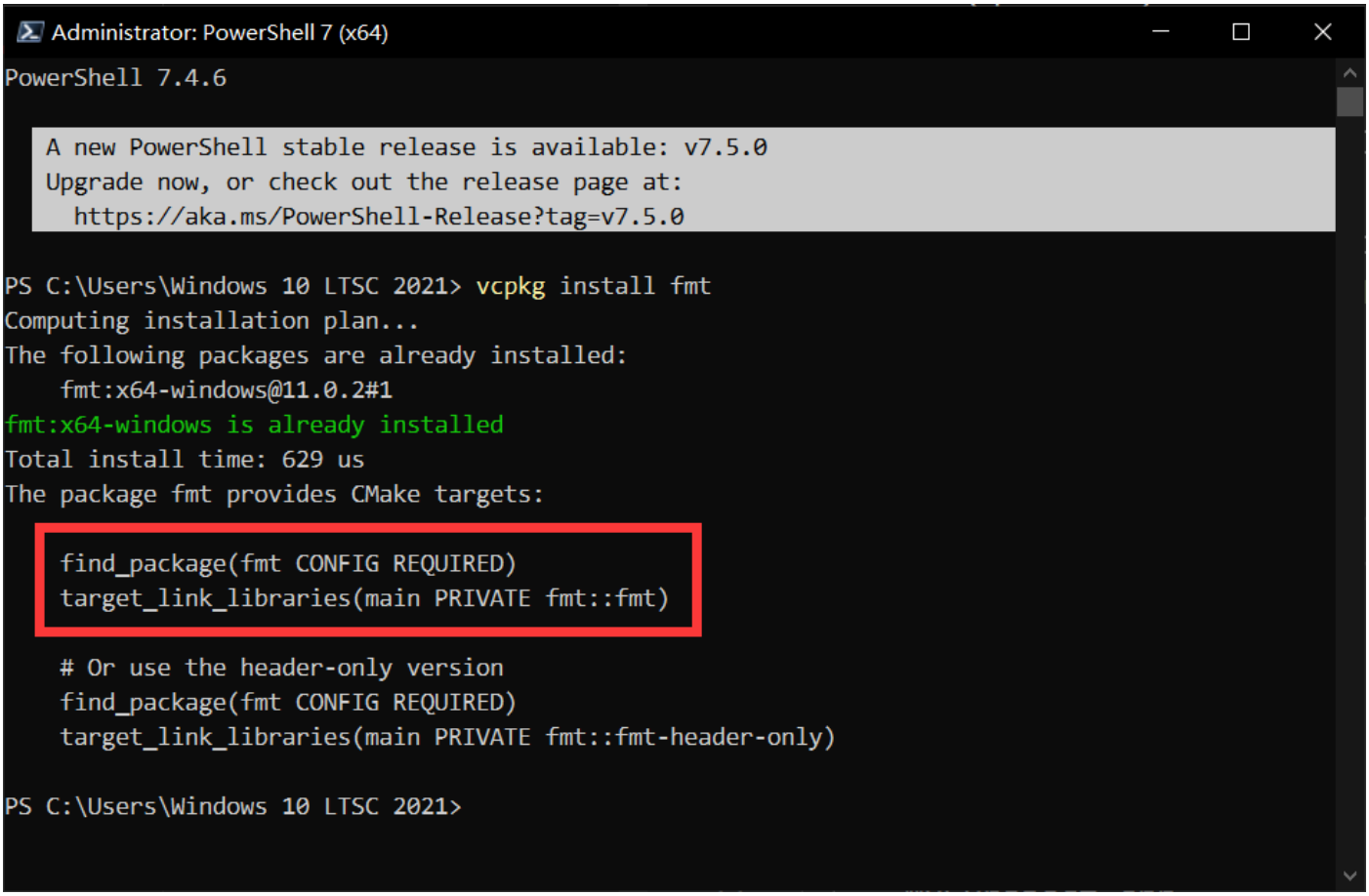
弹出 Qt Widgets Application Wizard，Next，Build System 选择 CMake Project for Qt，Next，Base class 根据需求选择，此处以 QWidget 为例，Finish。



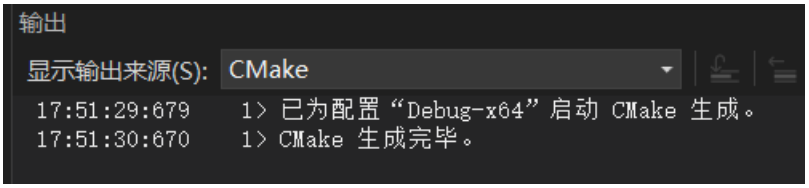
打开 CMakeUserPresets.json，设置 CMAKE_TOOLCHAIN_FILE 为 vcpkg.cmake 的路径并保存。



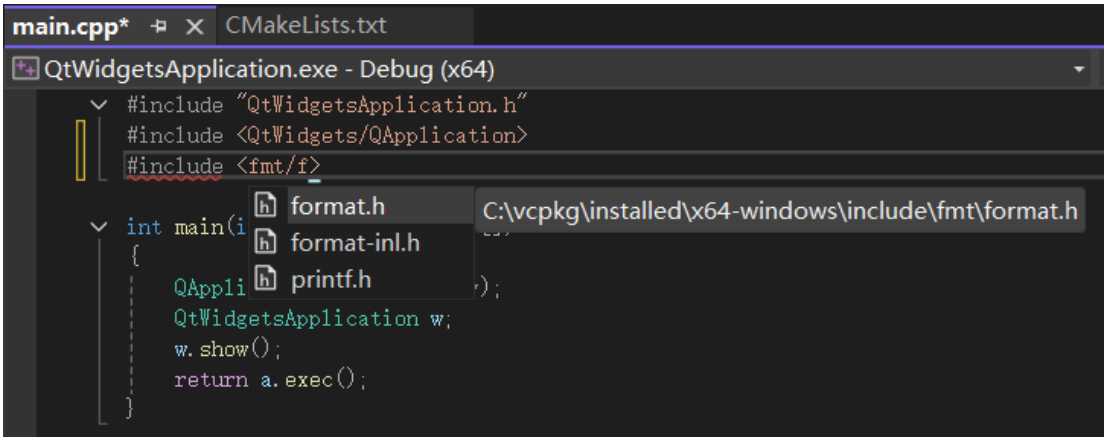
打开 CMakeLists.txt，运行桌面的 PowerShell 7 (x64)，输入“vcpkg install fmt”并执行。Ctrl + S 保存并生成 CMake 工具链。



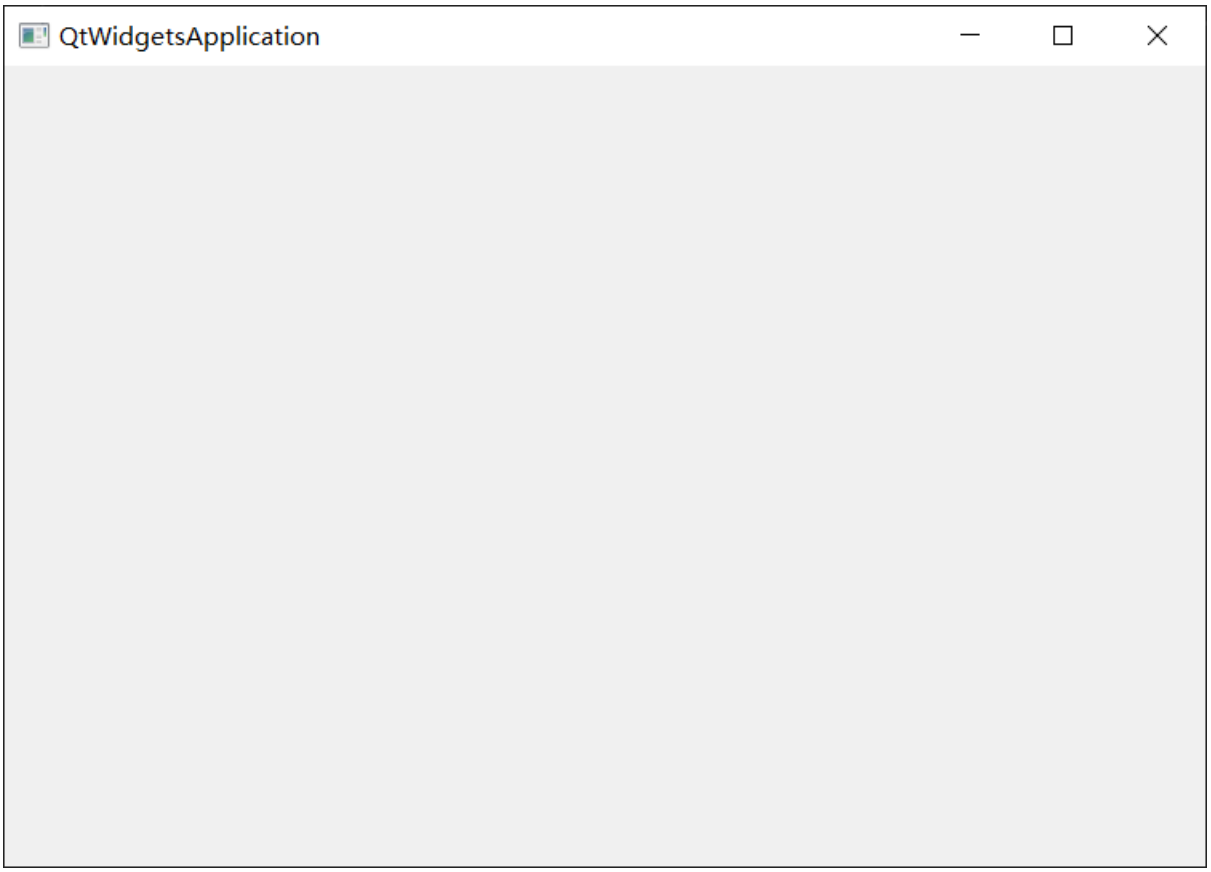
将 find_package(fmt CONFIG REQUIRED) 指令复制到 CMakeList.txt 的 find_package 旁，将 target_link_libraries(main PRIVATE fmt::fmt) 指令复制到 CMakeList.txt 对应指令旁，注意将指令中的 main 更改为你的项目名称，保存 CMakeList.txt 并生成 CMake 工具链。



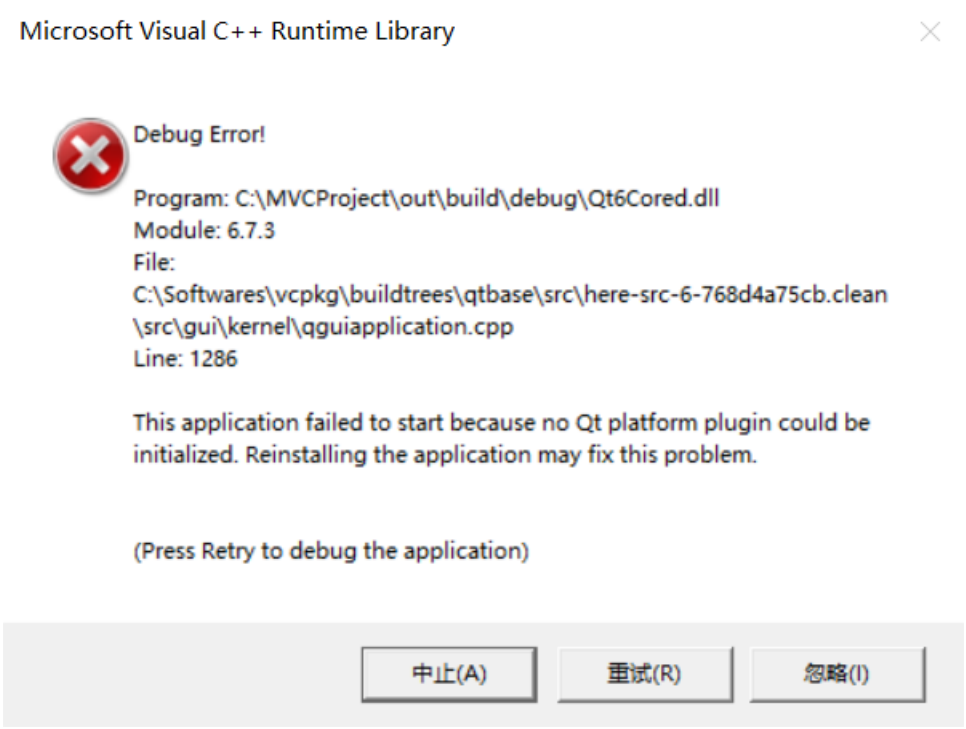
#include 可以自动侦测到 fmt 库相关 .h 文件。



运行程序可以弹出窗口：



如果遇到报错 This application failed to start because no Qt platform plugin could be initalized：



则在界面项目的 CMakeLists.txt 的末尾增加：

```
add_custom_command(
```

```
    TARGET main POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_directory
        ${Qt6_DIR}/../../$<$<CONFIG:Debug>:debug/>Qt6/plugins/platforms/
        $<TARGET_FILE_DIR:main>/platforms/
)
```

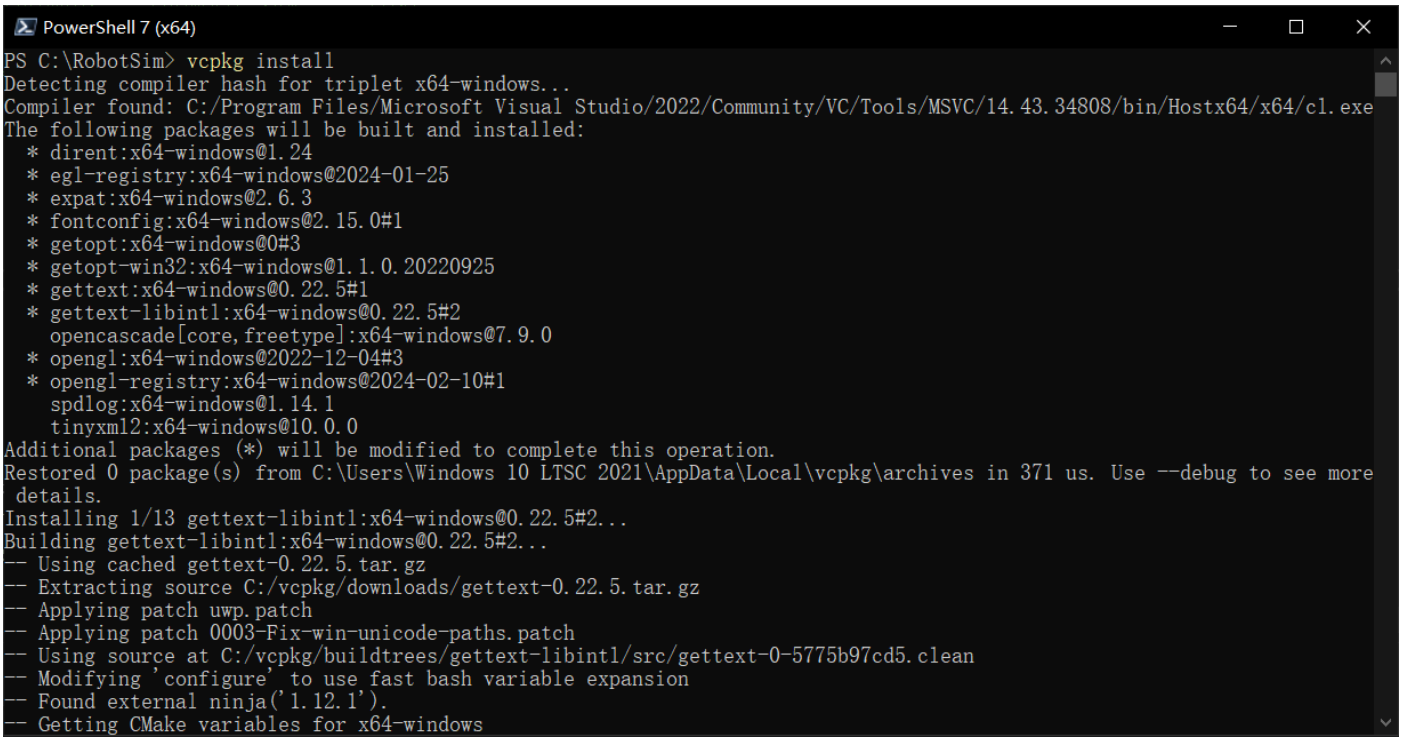
注意将 `main` 替换为界面项目的名称。

0x0A 测试二：打开 RobotSim 项目

解压代码包到工作目录。

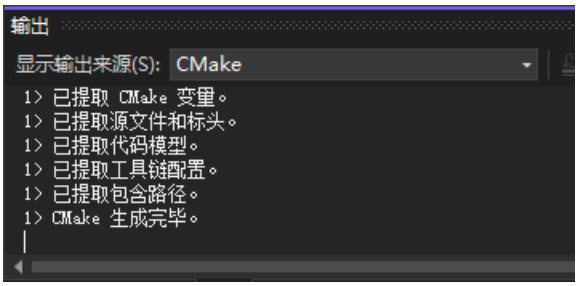


在工作目录运行 PowerShell，执行 vcpkg install，将自动下载编译所需要的库，初次运行需要等待较长时间。



运行 Visual Studio 2022，打开本地文件夹，选择工作目录，直到出现“CMake 生成

完毕”。


输出
显示输出来源(S): CMake
1> 已提取 CMake 变量。
1> 已提取源文件和标头。
1> 已提取代码模型。
1> 已提取工具链配置。
1> 已提取包含路径。
1> CMake 生成完毕。

编译并运行。

0x0B 测试三：创建 vcpkg 新项目

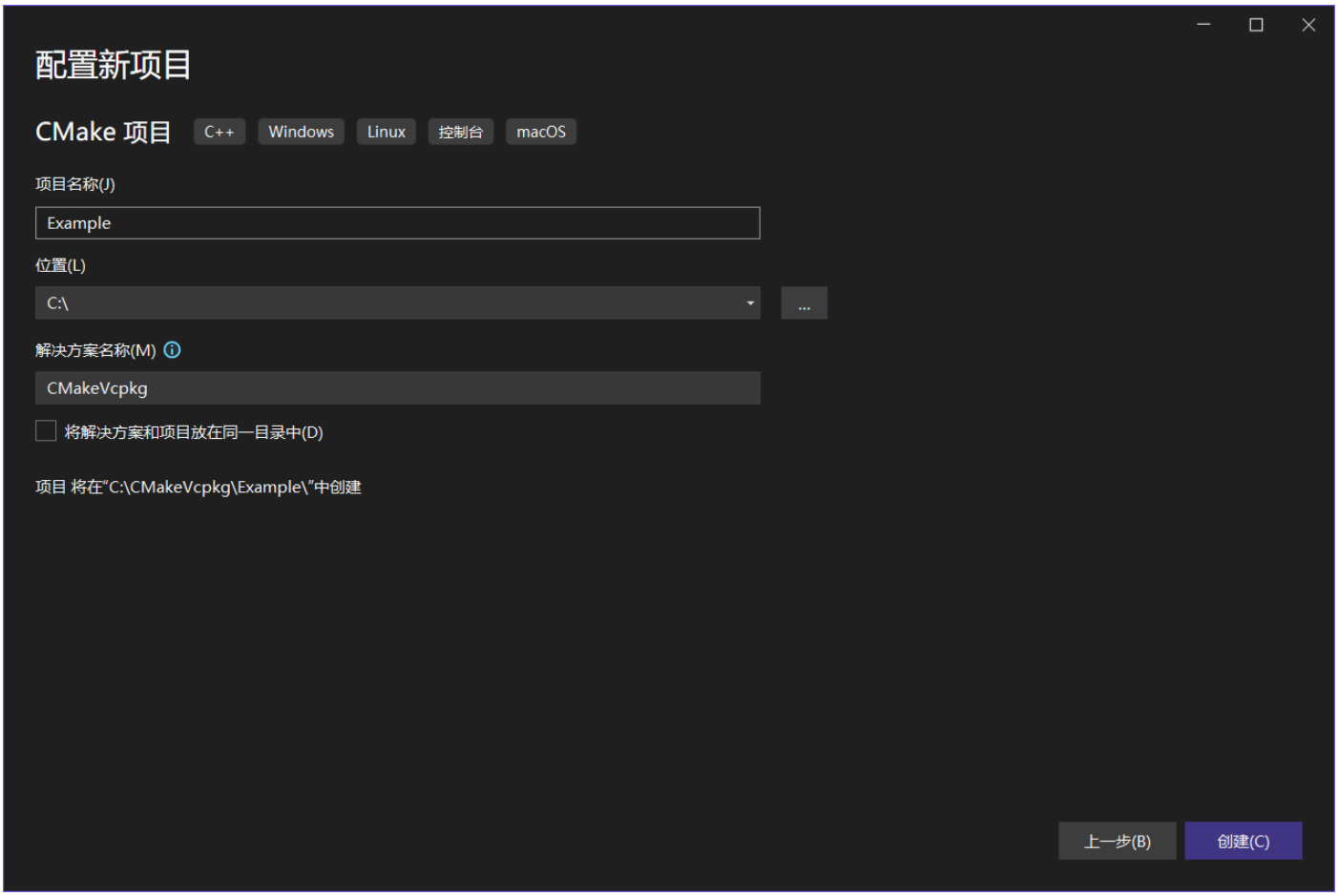
运行 Visual Studio 2022，创建新项目，CMake 项目，设定项目名称与位置并创建项目。

注意，若你的程序由多个子项目组成，每一个子项目有单独的构建目标（动态库、静态库或可执行文件），请取消勾选“将解决方案和项目放在同一目录中”，此时“解决方案名称”为整个大项目的名称，“项目名称”为第一个子项目的名称。

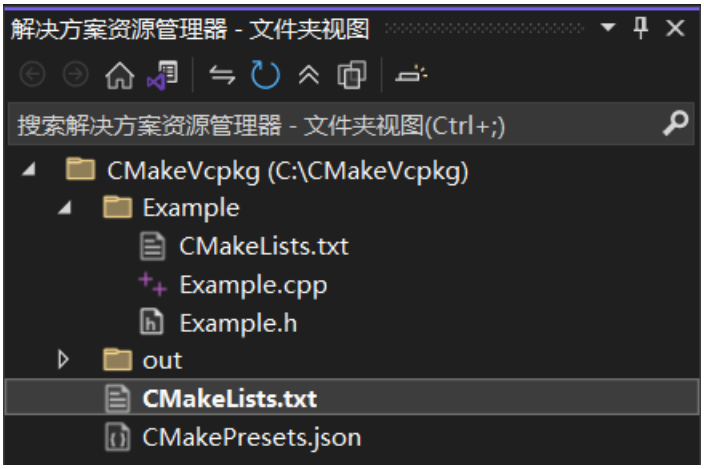
若你的项目不分子项目，所有代码为同一个构建目标，那么可以勾选“将解决方案和项目放在同一目录中”，此时“项目名称”为整个大项目的名称。

除非你确定你的项目的文件数量不超过 10 个，否则建议取消勾选“将解决方案和项目放在同一目录中”，本文以下图设定为例。

注意项目名请勿包含除了数字、大小写字母和下划线以外的字符，请勿包含空格。

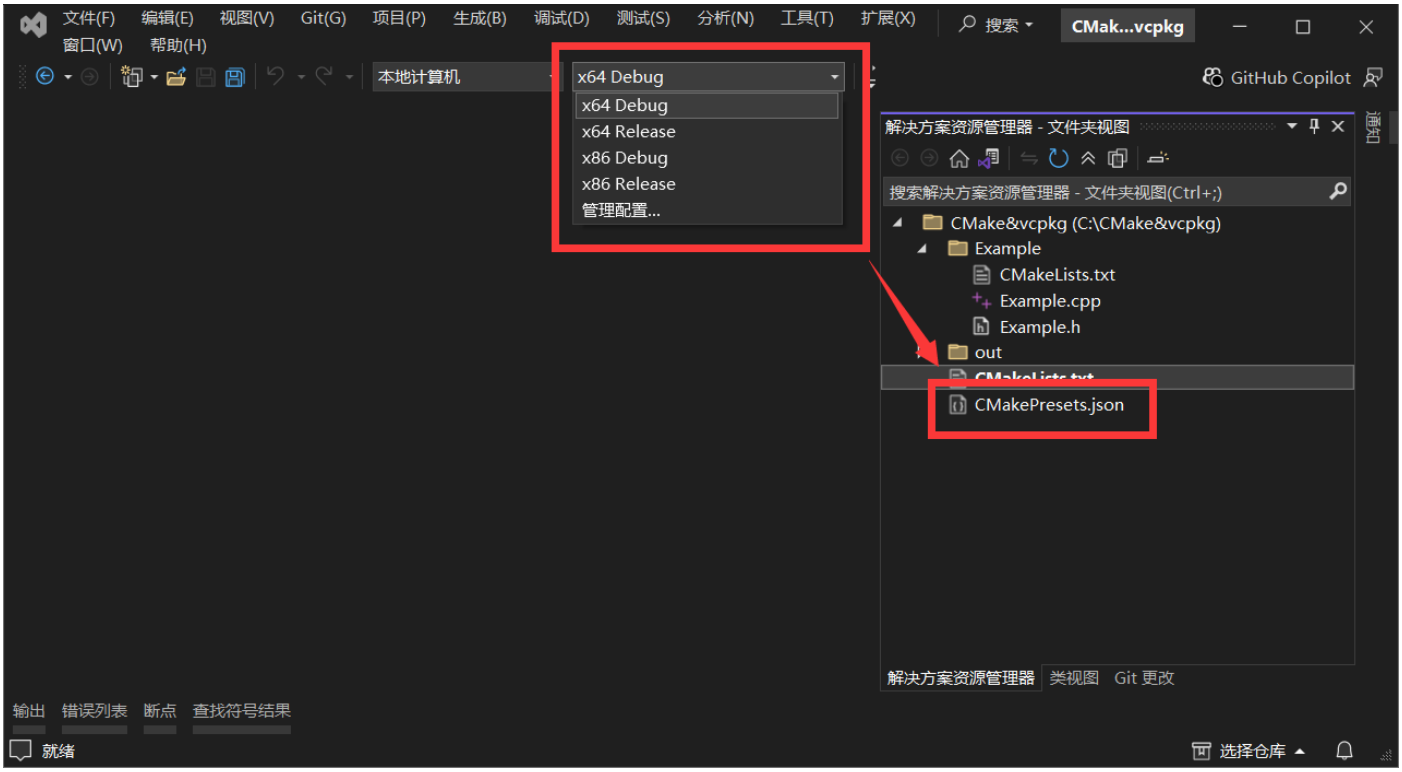


项目初始状态由 Visual Studio 生成了许多文件。



其中 `./out/` 文件夹用于存放 Visual Studio 的编译中间产物与最终文件，用户对其的更改会在下次编译时被覆盖，因此不应该更改同时也不能更改。

其中 `./CMakePresets.json` 用于配置构建目标平台与使用工具链，其中 `./` 意为当前文件夹。考虑到实验室大概率不会生成 32 位应用程序，因此我们可以删除 "x86-debug" 与 "x86-release" 两个 json 对象。



为了使 CMake 能联动 vcpkg 工具链，需要在 "windows-base" 对象的 "cacheVariables" 中新增 "CMAKE_TOOLCHAIN_FILE": "C:/vcpkg/scripts/buildsystems/vcpkg.cmake"，注意此处要替换为你的 vcpkg.cmake 的路径：

```
{
    "version": 3,
    "configurePresets": [
        {
            "name": "windows-base",
            "hidden": true,
            "generator": "Ninja",
            "binaryDir": "${sourceDir}/out/build/${presetName}",
            "installDir": "${sourceDir}/out/install/${presetName}",
            "cacheVariables": {
                "CMAKE_C_COMPILER": "cl.exe",
                "CMAKE_CXX_COMPILER": "cl.exe",
                "CMAKE_TOOLCHAIN_FILE": "C:/vcpkg/scripts/buildsystems/vcpkg.cmake"
            },
            "condition": {
                "type": "equals",
                "lhs": "${hostSystemName}",
                "rhs": "Windows"
            }
        },
        {
            "name": "x64-debug",
            "displayName": "x64 Debug",
            "inherits": "windows-base",
            "architecture": {
                "value": "x64",
                "strategy": "external"
            },
            "cacheVariables": {
                "CMAKE_BUILD_TYPE": "Debug"
            }
        },
        {
            "name": "x64-release",
            "displayName": "x64 Release",
            "inherits": "x64-debug",
            "cacheVariables": {
                "CMAKE_BUILD_TYPE": "Release"
            }
        }
    ]
}
```

保存并重新生成 Cmake 缓存。

./CmakeLists.txt 是整个大项目的 CMake 配置文件，可以按照下文配置：

```
cmake_minimum_required(VERSION 3.16)            # 指定 cmake 最低版本为 3.16

set(CMAKE_CXX_STANDARD 17)                      # 默认指定 C++ 17 标准
```

```
set(CMAKE_CXX_STANDARD_REQUIRED ON)             # 若不支持则不进行生成

set(TARGET_NAME "CMakeVcpkg")                   # 指定项目名称为 CMakeVcpkg
project(${TARGET_NAME} LANGUAGES CXX)           # 指定项目语言为 C++

if(MSVC)
    add_compile_options($<$<CXX_COMPILER_ID:MSVC>:/MP>) # 启用多线程编译

    # 设置 C 和 C++ 编译器优化选项
    set(CMAKE_C_FLAGS_DEBUG "/Od /ZI /MDd")     # Debug 模式: 禁用优化，启用调试信息，启用热重载
    set(CMAKE_CXX_FLAGS_DEBUG "/Od /ZI /MDd")   # Release 模式: 启用 LTO 优化和 O3 级别优化
    set(CMAKE_C_FLAGS_RELEASE "/Ox /Ob3 /MD")
    set(CMAKE_CXX_FLAGS_RELEASE "/Ox /Ob3 /MD")


    if(CMAKE_BUILD_TYPE MATCHES "Release")      # 配置 LTO 标志
        set(CMAKE_INTERPROCEDURAL_OPTIMIZATION_RELEASE ON)
        add_compile_options(/GL)
        add_link_options(/LTCG)
    endif()

    find_package(OpenMP REQUIRED)
    if(OpenMP_FOUND)                            # 尝试启用 OpenMP 多线程
        add_compile_options(${OpenMP_CXX_FLAGS})
        add_link_options(${OpenMP_EXE_LINKER_FLAGS})
    endif()
endif()

add_subdirectory("Example")                     # 编译各个项目
```

其中 set(TARGET_NAME "CMakeVcpkg") 设定大项目名，根据需求进行修改，建议大项目与文件夹名保持一致。

其中 add_subdirectory("Example") 用于添加子项目，需要与子项目的 CMakeLists.txt 中设定的子项目名保持一致。新增子项目之后请务必使用 add_subdirectory 语句添加。

为了通过 vcpkg 管理项目依赖的第三方库，我们需要在 ./ 下新建文件 vcpkg.json，以 fmt 库——一个开源的格式化输入输出库——为例，我们需要填入：

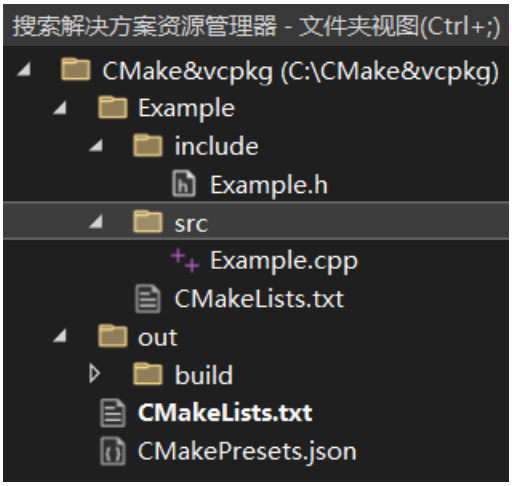
```
{
    "dependencies": [
        { "name": "fmt" }
    ]
}
```

保存文件，Visual Studio 工具栏，项目，删除缓存并重新配置。



如果操作得当，Visual Studio 会自动从 github 下载源码并构建 fmt 库，请注意是否因网络原因导致下载失败并耐心等待。直到 CMake 输出“CMake 生成完毕”则完成了 fmt 库的安装。

./Example/ 文件夹是子项目 Example 的文件夹，Visual Studio 自动创建了 Example.h 和 Example.cpp，但是分开 .h 与 .cpp 文件是更好的实践，因此在 ./Example/ 文件夹下新增 include 文件夹与 src 文件夹分别用于存放 .h 与 .cpp 文件。



./Example/CMakeLists.txt 为子项目 Example 的 CMake 配置文件，可以按照下文配置：

```
cmake_minimum_required(VERSION 3.16)    # 指定 cmake 最低版本为 3.16

set(CMAKE_CXX_STANDARD 23)              # 指定 C++ 23 标准
```

```
set(CMAKE_CXX_STANDARD_REQUIRED ON)     # 不支持则不进行生成

set(TARGET_NAME "Example")              # 指定项目名称为 Example
project(${TARGET_NAME} LANGUAGES CXX)   # 指定项目语言为 C++

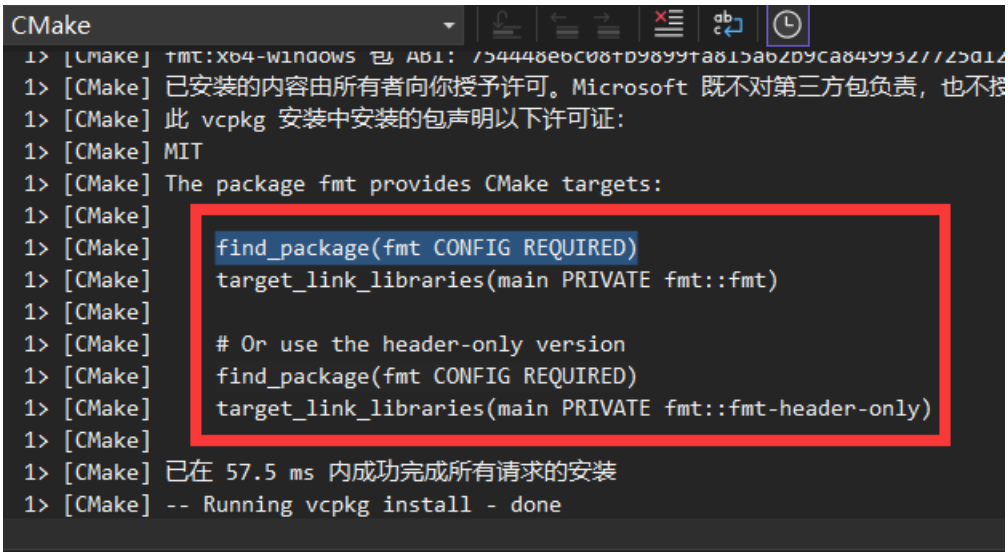
# 查找依赖，若不存在则不进行生成
find_package(fmt CONFIG REQUIRED)

# 生成名为 ${TARGET_NAME} 的静态库
file(GLOB HEADER_FILE "${CMAKE_CURRENT_SOURCE_DIR}/include/*.*")
file(GLOB SOURCE_FILE "${CMAKE_CURRENT_SOURCE_DIR}/src/*.*")
add_executable(${TARGET_NAME} ${SOURCE_FILE} ${HEADER_FILE})

# 设置头文件目录
# 这样 src/abc.cpp 就能直接 #include "abc.h" 而不是 "../include/abc.h"
target_include_directories(${TARGET_NAME}
    PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_link_libraries(${TARGET_NAME}    # 链接各种库
    # 用 vcpkg 安装库时，最后命令行会说如何在 CMake 中调用
    PRIVATE fmt::fmt
)
```

其中 find_package 用于查找外部依赖包。对于通过 vcpkg 管理的第三方库，在每次保存 CMakeList.txt 或 vcpkg.json，Visual Studio 自动配置环境安装依赖生成 CMake 缓存时，Cmake 通常会提示如何使用，通常由一句 find_package 和一句 target_link_libraries 组成。



其中 find_package 要写在 add_executable 之前，target_link_libraries 要写在 add_executable 之后。

其中 file 用于分别收集 include 文件夹中的所有文件和 src 文件夹中的所有文件，每次在 include 文件夹或 src 文件夹新增文件时，需要重新保存 CMakeLists.txt，重新收集文件并生成 CMake 缓存。

其中 add_executable 用于基于 HEADER_FILE 和 SOURCE_FILE 生成可执行文件，如果要生成静态库，可更改为 add_library(${TARGET_NAME} STATIC ${SOURCE_FILE} ${HEADER_FILE})。

其中 target_include_directories 用于公开头文件目录。

PRIVATE 模式下，头文件目录仅公开给当前子项目，同一个子项目中，./Example/src/Example.cpp 希望包含 ./Example/include/Example.h，可以直接 #include “Example.h” 而不用 #include “../include/Example.h”，其中 ../ 意为上级文件夹。

但是，跨项目的包含，例如子项目 UserInterface 依赖于子项目 Example，其中 ./UserInterface/src/ui.cpp 希望包含 ./Example/include/Example.h，只能使用相对路径，#include “../../Example/include/Example.h”。

PUBLIC 模式下，头文件目录公开给所有依赖于此子项目的项目，例如子项目 UserInterface 依赖于子项目 Example，其中 ./UserInterface/src/ui.cpp 希望包含 ./Example/include/Example.h，可以直接使用 #include “Example.h”。

但是，如果子项目 UserInterface 同时依赖于子项目 Example 和子项目 Fun，而 Example 与 Fun 同时公开了自己的头文件目录，并且子项目 Fun 中也有一个 Example.h，那么此时 ./UserInterface/src/ui.cpp 中使用 #include “Example.h”，就不能预知到底是包含了 ./Example/include/Example.h 还是 ./Fun/include/Example.h，造成预料以外的错误。

以上两种模式需要使用者进行取舍。

target_link_libraries 用于将当前构建目标与第三方库链接，同样存在 PRIVATE 模

式与 PUBLIC 模式。

PRIVATE 模式意为该库只允许当前子项目链接，如果当前项目的所有 .h 文件都完全没有包含这个库的头文件，或者使用前向声明规避了包含，那么可以使用 PRIVATE 模式。

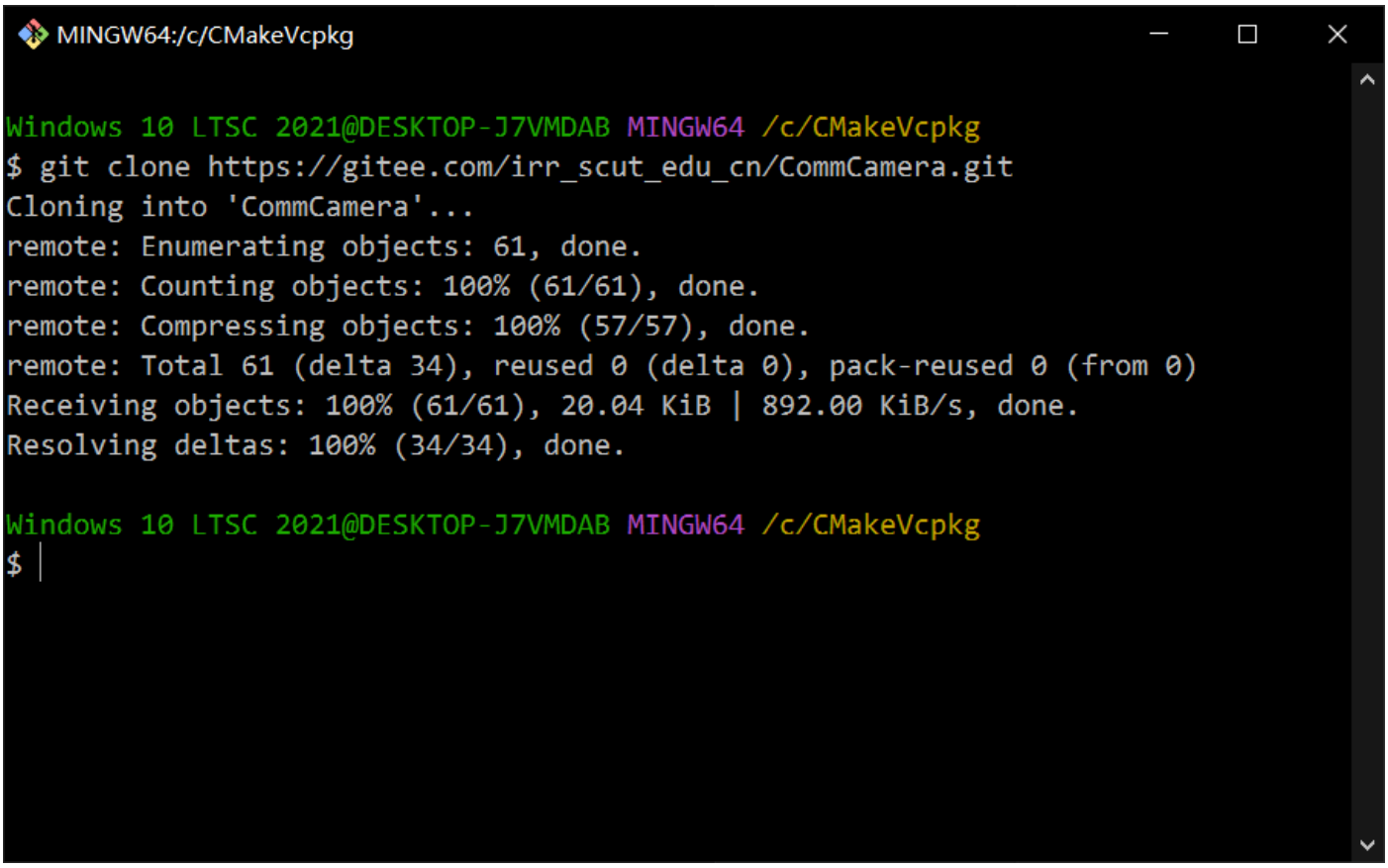
PUBLIC 模式意为该库允许当前子项目、依赖于当前子项目的子项目、依赖于“依赖于当前子项目的子项目”的子项目等等等等子项目链接该第三方库。在当前子项目的所有 .h 文件中，只要有一处包含了该第三方库的头文件，那么就需要使用 PUBLIC 模式。

任何关于 CMakeLists.txt 如何编写的问题都可以咨询 AI。

在 ./Example/src/Example.cpp 中新增 #include <fmt/format.h> 和 fmt::print("Hello fmt.\n");，编译运行，控制台成功打印即视为配置完成。

0x0C 测试四：调用相机通讯库 CommCamera

以上文的 CMakeVcpkg 项目为例，在 ./ 打开 git 控制台，使用 git clone https://gitee.com/irr_scut_edu_cn/CommCamera.git 从远程仓库复制相机通讯库 CommCamera 的代码到本地，此时 CommCamera 库成为大项目 CMakeVcpkg 的一个子项目。



CommCamera 库也通过 vcpkg 管理第三方依赖库，其第三方依赖写在在 ./CommCamera/vcpkg.json 中，但是只有大项目中的 ./vcpkg.json 会被识别，因此需要将 ./CommCamera/vcpkg.json 的内容合并进 ./vcpkg.json。

```
{
    "dependencies": [
        { "name": "fmt" },
        { "name": "magic-enum" },
        { "name": "opencv4" }
    ]
}
```

在编译子项目 CommCamera 之前，请根据需要使用到的相机品牌安装对应的 SDK。目

前 CommCamera 支持巴斯勒、海康和大恒相机，如果你的电脑没有安装某品牌相机的 SDK，那么请在 ./CommCamera/CMakeLists.txt 中关闭对应的选项。

```
# 配置相机启用选项
set(COMM_CAMERA_BUILD_BASLER ON CACHE BOOL "Build with Basler camera support" FORCE)
set(COMM_CAMERA_BUILD_DAHENG OFF CACHE BOOL "Build with Daheng camera support" FORCE)
set(COMM_CAMERA_BUILD_HIKVISION ON CACHE BOOL "Build with Hikvision camera support" FORCE)
```

以未安装大恒 SDK 为例，此时需要将 COMM_CAMERA_BUILD_DAHENG 标志位设置为 OFF。

现在大项目 CMakeVcpkg 新增了子项目 CommCamera，不要忘记在 ./CMakeLists.txt 中新增 add_subdirectory("CommCamera")。

```
add_subdirectory("Example")                    # 编译各个项目
add_subdirectory("CommCamera")
```

我们希望子项目 Example 可以调用子项目 CommCamera，因此在 ./Example/CMakeLists.txt 中新增 target_link_libraries：

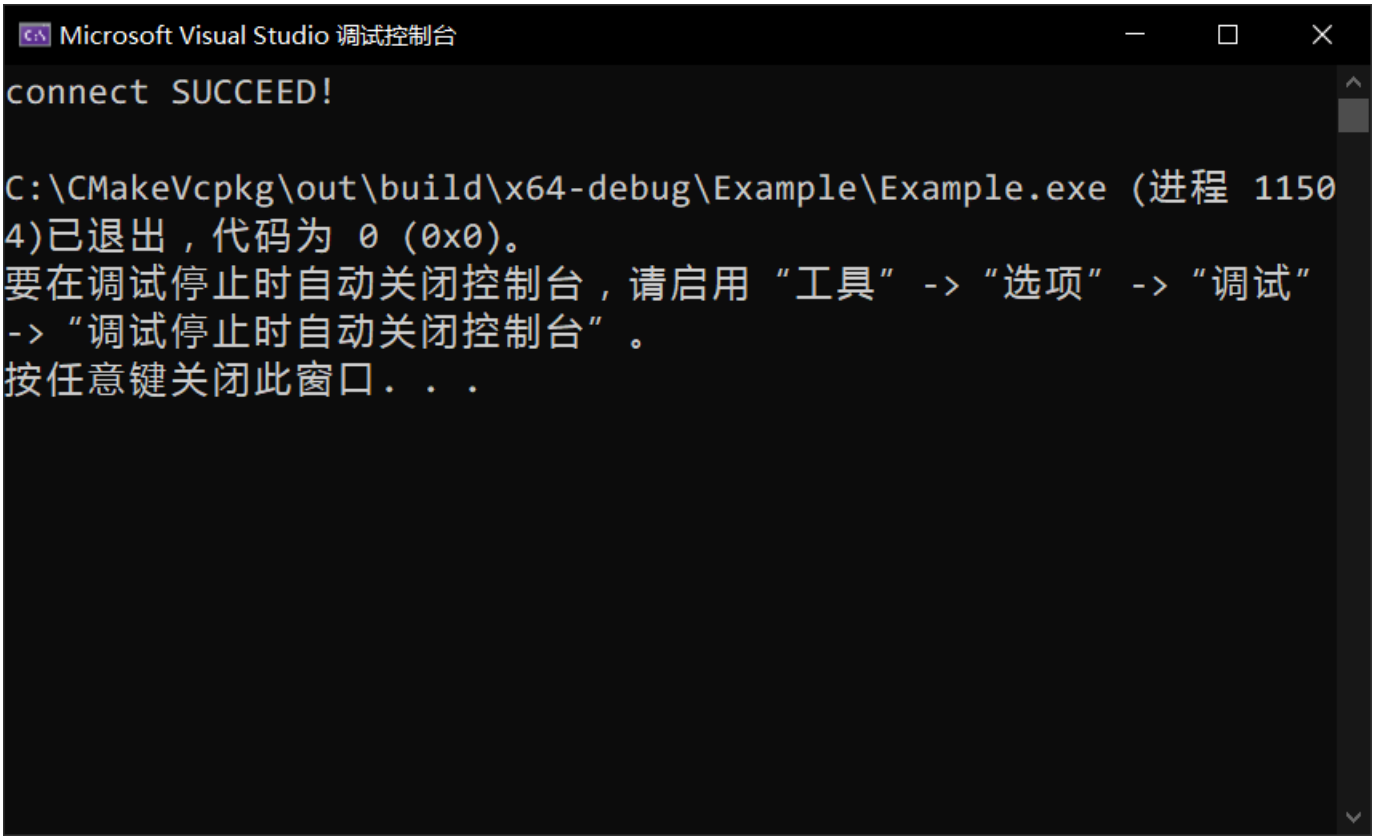
```
target_link_libraries(${TARGET_NAME}     # 链接各种库
    # 用 vcpkg 安装库时，最后命令行会说如何在 CMake 中调用
    PRIVATE fmt::fmt
    PRIVATE CommCamera
)
```

以连接海康相机为例，在 ./Example/src/Example.cpp 替换代码为：

```
#include <fmt/format.h>
#include "../../CommCamera/include/CameraHik.h"

int main()
{
    i2r_comm_camera::CameraHik camera;
    if (auto result = camera.connect()) fmt::print("connect SUCCEED!\n");
    else fmt::print("connect FAILED: {}", static_cast<unsigned int>(result.error()));
    return 0;
}
```

连接相机，运行程序，显示 connect SUCCEED! 即连接成功。


Microsoft Visual Studio 调试控制台
connect SUCCEED!
C:\CMakeVcpkg\out\build\x64-debug\Example\Example.exe (进程 11504)已退出，代码为 0 (0x0)。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口. . .

```
0xFF
```

以上流程经过测试验证可行。